DA2 Report Content: SQL and PL/SQL Implementation

Project: Satellite Imagery Knowledge Graph (GraphRAG for Earth Observation data) **DBMS:** Oracle Database 21c Express Edition · **Tool:** Oracle SQL Developer 26.2 · **Schema owner:** JOSHWA **Scripts** (in `database/`, run in this order): `schema.sql → sample_data.sql → plsql.sql → run_procedures.sql → queries.sql → extras.sql`

---

1. Aim

Implement the DA1 design (15 relations, BCNF) as a working Oracle database, and demonstrate:

1. table creation with all integrity constraints
2. realistic sample records
3. SQL queries, from simple to advanced
4. PL/SQL programs: functions, a package, procedures, cursors, exception handling and triggers
5. transaction control, DDL changes and access control (DCL)

2. Environment Setup

1. Oracle 21c XE was installed. The pluggable database `XEPDB1` holds the project schema.
2. User **JOSHWA** was given these privileges (by SYS): `GRANT CREATE VIEW, CREATE PROCEDURE, CREATE TRIGGER, CREATE SEQUENCE TO JOSHWA;` The first run failed with **ORA-01031: insufficient privileges** while creating the views; this grant fixed it.
3. SQL Developer connection: host `localhost`, port `1521`, service name `XEPDB1`.
4. Each script was run with **Run Script (F5)**, and individual queries with **Ctrl+Enter**.

---

3. Table Creation (`schema.sql`)

Result: 15 tables, 12 indexes and 2 views were created with no errors. The script first drops any existing project tables, so it can be re-run safely.

3.1 Example DDL (write 2–3 of these in the report)

sql

```sql
CREATE TABLE image_frame (
  image_id         NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY (START WITH 10000),
  sensor_id        NUMBER        NOT NULL,
  location_id      NUMBER        NOT NULL,
  capture_time     TIMESTAMP     NOT NULL,
  cloud_cover      NUMBER(5,2)   NOT NULL,
  sun_elevation    NUMBER(5,2),
  file_path        VARCHAR2(250) NOT NULL,
  image_embedding  CLOB,
  image_created_at TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
  CONSTRAINT pk_image_frame PRIMARY KEY (image_id),
  CONSTRAINT fk_img_sensor FOREIGN KEY (sensor_id)   REFERENCES sensor(sensor_id),
  CONSTRAINT fk_img_loc    FOREIGN KEY (location_id) REFERENCES location(location_id),
  CONSTRAINT uq_img_capture UNIQUE (sensor_id, location_id, capture_time),
  CONSTRAINT uq_img_path    UNIQUE (file_path),
  CONSTRAINT ck_img_cloud CHECK (cloud_cover BETWEEN 0 AND 100),
  CONSTRAINT ck_img_sun   CHECK (sun_elevation BETWEEN 0 AND 90),
  CONSTRAINT ck_img_emb   CHECK (image_embedding IS JSON)
);
```

sql

```sql
CREATE TABLE object_change (            -- EVOLVED_FROM lineage (the novelty)
  change_id          NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY (START WITH 10000),
  prev_object_id     NUMBER       NOT NULL,
  curr_object_id     NUMBER       NOT NULL,
  change_type        VARCHAR2(20) NOT NULL,
  change_pct         NUMBER(8,2)  NOT NULL,
  time_delta_days    NUMBER(5)    NOT NULL,
  rate_pct_per_month NUMBER(8,2)  NOT NULL,
  computed_at        TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
  CONSTRAINT pk_object_change PRIMARY KEY (change_id),
  CONSTRAINT fk_chg_prev FOREIGN KEY (prev_object_id) REFERENCES detected_object(object_i
  CONSTRAINT fk_chg_curr FOREIGN KEY (curr_object_id) REFERENCES detected_object(object_i
  CONSTRAINT uq_chg_pair UNIQUE (prev_object_id, curr_object_id),
  CONSTRAINT ck_chg_diff CHECK (prev_object_id <> curr_object_id),
  CONSTRAINT ck_chg_type CHECK (change_type IN ('DEFORESTATION','GRADUAL_LOSS','STABLE','
);
```

3.2 Design decisions in the DDL

Decision	Reason
`GENERATED BY DEFAULT ON NULL AS IDENTITY (START WITH 10000)`	automatic surrogate keys for new rows; sample data uses IDs 1–9999, so they never clash
Natural keys kept as `UNIQUE` (satellite_name, basin_code, file_path, …)	preserves the candidate keys from normalization
Embeddings and attributes as `CLOB` + `CHECK (… IS JSON)`	Oracle XE has no vector type; the JSON check guarantees valid data
`TIMESTAMP DEFAULT SYSTIMESTAMP`	records when each row was created
Every FK column indexed (12 indexes)	faster joins, and avoids table locks when deleting a parent row

3.3 Views

View	Purpose
`v_image_summary`	image + sensor + satellite + location + basin in one row (a 5-table join reused by queries and the app)
`v_forest_change`	each lineage change with its location, basin, both images' cloud cover and both detections' confidence

4. Constraints Implemented

Type	Count	Examples
PRIMARY KEY	15	`pk_satellite`, `pk_detected_object`
FOREIGN KEY	17	`fk_obj_image` (CASCADE), `fk_loc_basin` (SET NULL), `fk_img_sensor` (restrict)
UNIQUE (candidate keys)	17	`uq_satellite_name`, `uq_sensor (satellite_id, sensor_name)`, `uq_si (image_id, index_name)`, `uq_alert_change`
NOT NULL	most columns	all FKs with total participation
CHECK: ranges	many	`cloud_cover 0–100`, `confidence 0–1`, `lat −90…90`, `area > 0`, `rank > 0`
CHECK: enumerations	10	orbit_type, sensor_type, region_type, change_type, severity, alert_status, user_role
CHECK: multi-column	several	bbox `min < max`; `min ≤ mean ≤ max` for spectral indices; `prev ≠ curr`; `resolved_at` only when RESOLVED
CHECK: format	5	`IS JSON` on embeddings, attributes and weights; email `LIKE '%_@_%._%'`
DEFAULT	several	`sat_status 'ACTIVE'`, `alert_status 'OPEN'`, timestamps

Referential actions:

- `ON DELETE CASCADE`: the child has no meaning without its parent (sensor → satellite, object → image, index → image, change/evidence → object, alert → change).
- `ON DELETE SET NULL`: the link is optional (location → basin, query → user, alert → user).
- No action (restrict): deleting a sensor that still has images is refused, so history is never lost silently.

---

5. Sample Records (`sample_data.sql`)

The data was generated by a deterministic Python script (`scripts/generate_data.py`), so the SQL data and the Neo4j CSVs (DA3) always match.

Table	Rows	Content
SATELLITE	5	Sentinel-2A/2B, Landsat 8/9, Sentinel-1A
SENSOR	6	MSI (10 m), OLI/OLI-2 (30 m), TIRS (thermal), C-SAR
RIVER_BASIN	5	Amazon, Congo, Mekong, Godavari, Danube
LOCATION	12	9 inside basins, 3 outside (Chennai Coast, Rotterdam Port, Dubai Marina)
OBJECT_TYPE	8	Tree Coverage, Building, Vessel, Road, Water Body, Cropland, Bare Soil, Wildfire Scar
DETECTION_MODEL	3	ForestNet 1.2, YOLO-EO 8.0, NDVI-Threshold 2.0
APP_USER	4	2 analysts, 1 admin, 1 viewer
IMAGE_FRAME	60	12 locations × 5 dates (Apr, Jun, Jul, Aug, Sep 2026)
DETECTED_OBJECT	247	one tree patch per forest image + other objects
SPECTRAL_INDEX	120	NDVI + NDWI per image
QUERY_LOG	4	logged questions with weights α/β/γ
QUERY_EVIDENCE	20	top-5 evidence per question

Designed patterns in the data:

- Four forest locations **deforest rapidly** (Manaus, Kisangani, Tonle Sap, Bhadrachalam).
- Three **lose forest slowly** (Rondonia, Kratie, Rajahmundry).
- Two are **stable** (Mbandaka, Belgrade).

OBJECT_CHANGE, ALERT, AUDIT_LOG and `query_log.aggregated_confidence` are **not inserted directly**. They are produced by the PL/SQL programs.

---

6. PL/SQL Programs (`plsql.sql`)

Result: 3 functions, 1 package spec, 1 package body and 4 triggers compiled with **No errors**.

6.1 Stand-alone functions

Function	What it does	Used in
`fn_haversine_km(lat1, lon1, lat2, lon2)`	great-circle distance in km (DETERMINISTIC)	"near a river" filter (Q10, app finder)
`fn_classify_change(pct)`	returns DEFORESTATION (≥10%), GRADUAL_LOSS (≥3%), STABLE, REGROWTH (≤−10%)	`compute_changes`
`fn_forest_loss_pct(location, from, to)`	% tree-area loss between the first and last image in a date window, using `KEEP (DENSE_RANK FIRST/LAST)`	basin report, app dashboard

6.2 Package `pkg_eo` (encapsulates the project's business logic)

sql

```sql
CREATE OR REPLACE PACKAGE pkg_eo AS
  e_no_evidence EXCEPTION;
  PRAGMA EXCEPTION_INIT(e_no_evidence, -20001);
  PROCEDURE compute_changes;
  PROCEDURE raise_alerts (p_min_loss NUMBER DEFAULT 10, p_min_conf NUMBER DEFAULT 0.7, p_
  FUNCTION  fn_query_confidence (p_query_id NUMBER) RETURN NUMBER;
  PROCEDURE finalize_query (p_query_id NUMBER);
  PROCEDURE add_detection (p_image_id NUMBER, p_type_name VARCHAR2, p_model_id NUMBER, p_
                           p_lat NUMBER, p_lon NUMBER, p_area NUMBER, p_new_id OUT NUMBER
  PROCEDURE basin_report (p_basin_code VARCHAR2);
END pkg_eo;
```

Member	PL/SQL features used	What it does
`compute_changes`	**explicit cursor** with the analytic `LAG() OVER (PARTITION BY location ORDER BY time)`, cursor FOR loop, **MERGE** (upsert)	builds the **EVOLVED_FROM lineage**: links each tree patch to the previous patch at the same location and stores change %, days and rate per month
`raise_alerts`	default parameters, implicit cursor loop, `NOT EXISTS`	raises an alert only when loss ⩾ 10% **and** both detections have confidence ⩾ 0.7 **and** both images have ⩽ 40% cloud (trust-aware alerting). Severity: ⩾25% CRITICAL, ⩾15% HIGH, else MEDIUM
`fn_query_confidence`	aggregate in `SELECT INTO`, `NULLIF`	**confidence propagation:** retrieval-weighted mean of (detection confidence × clear-sky fraction) over the question's evidence
`finalize_query`	`RAISE_APPLICATION_ERROR(-20001)`, `CASE`	stores the confidence and a generated answer sentence on QUERY_LOG
`add_detection`	**SAVEPOINT / ROLLBACK TO**, `RETURNING INTO`, OUT parameter, nested block with `NO_DATA_FOUND`, `DUP_VAL_ON_INDEX`, `VALUE_ERROR`, `OTHERS`	validated insert of a new detection
`basin_report`	**parameterised cursor** `c_loc(cp_code)`, BOOLEAN flag, `RPAD`	prints forest loss per location in a basin

6.3 Triggers

Trigger	Timing / level	Purpose
`trg_obj_audit`	AFTER INSERT/UPDATE/DELETE, **row-level** on DETECTED_OBJECT	writes old/new values to AUDIT_LOG, using `INSERTING`, `UPDATING`, `:OLD`, `:NEW`
`trg_img_validate`	BEFORE INSERT/UPDATE OF capture_time, row-level	rejects a capture time in the future (ORA-20010). A CHECK constraint cannot use SYSTIMESTAMP
`trg_alert_resolve`	BEFORE UPDATE OF alert_status, row-level	stamps `resolved_at` automatically when the status becomes RESOLVED, and clears it otherwise
`trg_change_validate`	BEFORE INSERT/UPDATE, row-level on OBJECT_CHANGE	both objects must be at the same location, and the previous one must be older (ORA-20030/20031)
`trg_alert_stmt_audit` (extras.sql)	AFTER INSERT/UPDATE/DELETE, **statement-level** on ALERT	one audit row per statement, however many rows changed

> **Issue found during testing:** the first version of `trg_alert_stmt_audit` used `CASE WHEN INSERTING …` inside the SQL INSERT. `INSERTING` is a PL/SQL-only boolean, so the trigger compiled **invalid**. Later statements touching ALERT then failed with **ORA-04098** (even a DELETE on DETECTED_OBJECT, because it cascades to ALERT). **Fix:** evaluate `INSERTING`/`UPDATING` in PL/SQL `IF` statements into a variable, then insert the variable.

7. Running the PL/SQL (`run_procedures.sql`): actual output

```
compute_changes: 36 lineage links created/updated.
raise_alerts: 4 new alerts.
finalize_query(1): Answer based on 5 evidence objects. Overall confidence 61.2%
(moderate - ...)
finalize_query(2): ... Overall confidence 66.6% (moderate ...)
finalize_query(3): ... Overall confidence 61.8% (moderate ...)
finalize_query(4): ... Overall confidence 60.1% (moderate ...)
=== Forest loss report, basin MKG (Apr-Sep 2026) ===
Kratie Riverbank            7.74 % loss
Tonle Sap Fringe            38.49 % loss
=== Forest loss report, basin AMZ (Apr-Sep 2026) ===
Manaus Forest Edge          38.72 % loss
Rondonia Frontier           8.13 % loss
Inserted detection id = 10000
Expected error caught: ORA-20020: Unknown object type: Spaceship
Expected error caught: ORA-20010: capture_time cannot be in the future: 01-JAN-30 ...
ORA-06512: at "JOSHWA.TRG_IMG_VALIDATE", line 3
```

Interpretation:

- **36 lineage links** = 9 forest locations × 4 consecutive image pairs.
- **4 alerts:** 10 changes qualify as DEFORESTATION, but only 4 had confident, clear-sky evidence on both images. The other 6 were filtered out as untrustworthy, which is the trust-aware design working.
- **Answer confidences** 0.6121 / 0.6655 / 0.618 / 0.6006 match the values predicted by the data generator to 4 decimals. This independently confirms that the PL/SQL formula is correct.
- **Basin report:** the rapidly deforesting sites lost about 38% over 6 months; the slow-loss sites about 8%.
- **Error handling:**
  - the package rejected an unknown object type (ORA-20020) and rolled back to its SAVEPOINT
  - the trigger rejected a future capture date (ORA-20010)

Result tables after the demo UPDATEs:

alert_id	severity	status	assignee	resolved_at
10000	HIGH	RESOLVED	1	filled automatically by trg_alert_resolve
10001	MEDIUM	OPEN	none	none
10002	MEDIUM	OPEN	none	none
10003	HIGH	OPEN	none	none

audit_id	table	operation	record	old_value	new_value
1	DETECTED_OBJECT	INSERT	10000	none	conf=.91, area=850
2	DETECTED_OBJECT	UPDATE	1	conf=.837, area=5949857.19	conf=.99, area=5949857.19

After this script the database holds **248** detected objects (247 sample + 1 demo insert). Re-running the script adds one more demo detection each time.

8. SQL Queries (`queries.sql`): purpose, concepts and actual results

Q	Question answered	SQL concepts	Result
Q1	first 10 images with satellite, sensor, location	view (5-table join), `FETCH FIRST`	10 rows; all 10-Apr-26 images, e.g. image 1 Manaus, Sentinel-2B MSI, 31.4% cloud
Q2	locations outside any basin	`LEFT JOIN … WHERE b.basin_id IS NULL` (anti-join)	3 rows: Chennai Coast, Rotterdam Port, Dubai Marina
Q3	clear-sky (< 20% cloud) images in Q3 2026	timestamp range, `ORDER BY`	12 rows; clearest is image 23, Tonle Sap, 3.7%
Q4	images and average cloud per satellite	`JOIN`, `GROUP BY`, `COUNT`, `AVG`	4 satellites × 15 images; avg cloud 26.5–32.5%. Sentinel-1A has no images, so the inner join drops it
Q5	detections per object type, with > 5 detections	`GROUP BY … HAVING`	7 types (Building 56, Tree 45, Water 40, Vessel 35, Road 25, Cropland 25, Bare Soil 20); Wildfire Scar removed by HAVING
Q6	objects per river basin	4-hop `LEFT JOIN` chain, aggregate	Godavari 45, Mekong 35, Amazon 33, Congo 30, Danube 20
Q7	images cloudier than average	**non-correlated subquery**	8 rows, cloudiest image 18 (Mbandaka, 53.3%)
Q8	latest tree area per location	**correlated subquery** (`MAX` per location)	9 rows; Kisangani has the smallest remaining forest (2.61 km²)
Q9	deforested locations with **no** alert	**MINUS** set operator	Manaus, Bhadrachalam: their evidence was too cloudy or uncertain, so no alert was raised
Q10	**"rapid deforestation near river basins in Q3"**	view + PL/SQL function in `WHERE`, multi-table join	**10 rows**: Manaus, Kisangani, Tonle Sap, Bhadrachalam; 38–174 km from the river; fastest rate 16.15%/month
Q11	the same events ranked by trust	`LEAST`, `GREATEST`, arithmetic expression	best Kisangani 29-Sep (0.584), worst Manaus 27-Aug (0.306)

Q	Question answered	SQL concepts	Result
Q12	cumulative loss and rank per period	**window functions** `SUM() OVER`, `RANK() OVER`	36 rows; e.g. Kisangani cumulative 13.87% by 22-Jul
Q13	full lineage chain of each forest patch	**hierarchical query** `CONNECT BY`, `SYS_CONNECT_BY_PATH`, `LEVEL`	36 rows; e.g. `1 → 3 → 5 → 7 → 9` (Manaus)
Q14	does NDVI agree with the tree area?	statistical aggregate `CORR`	0.442 (positive). Moderate because each location starts with a different forest size
Q15	vessels by class	**JSON functions** `JSON_VALUE … RETURNING NUMBER`	cargo 11 (avg 127.3 m), tanker 9 (160.7 m), passenger 8 (138.8 m), fishing 6 (101 m)
Q16	images by resolution class	`CASE` inside `GROUP BY`	High (≤10 m) 30 images, Medium (≤30 m) 30
Q17	top-3 evidence per logged question	6-table join	12 rows; question 1's top evidence is Kisangani (0.9484)
Q18	alerts dashboard	`LEFT JOIN`, `ORDER BY CASE` (custom sort)	4 alerts; 10000 HIGH Kisangani RESOLVED (Aarav Menon), 10003 HIGH Tonle Sap, 10002/10001 MEDIUM Tonle Sap
Q19	transaction demo	`SAVEPOINT`, `UPDATE`, `ROLLBACK TO`	"5 rows updated" → "Rollback complete"; data unchanged
Q20	integrity demos	invalid DML	ORA-02290 (cloud 140), ORA-02291 (missing parent image), ORA-02292 (delete a satellite with child rows)

The key query (Q10):

```sql
```

```sql
SELECT fc.location_name, fc.basin_name, fc.curr_time, fc.change_pct, fc.rate_pct_per_month
       ROUND(fn_haversine_km(l.latitude, l.longitude, b.river_ref_lat, b.river_ref_lon)) A
FROM v_forest_change fc
JOIN location l    ON l.location_id = fc.location_id
JOIN river_basin b ON b.basin_id = l.basin_id
WHERE fc.change_type = 'DEFORESTATION'
  AND fc.curr_time >= TIMESTAMP '2026-07-01 00:00:00'
  AND fc.curr_time <  TIMESTAMP '2026-10-01 00:00:00'
  AND fn_haversine_km(l.latitude, l.longitude, b.river_ref_lat, b.river_ref_lon) <= 300
ORDER BY fc.rate_pct_per_month DESC;
```

9. Additional Features (`extras.sql`)

Section	Feature	Expected output
E1	**DDL:** `ALTER TABLE` ADD column, MODIFY, ADD CONSTRAINT, RENAME COLUMN, DISABLE/ENABLE constraint, DROP column; `COMMENT ON`	satellites listed with mass_kg; CK_IMG_CLOUD status DISABLED then ENABLED; table comment shown; column dropped again
E2	**DCL:** `GRANT SELECT/INSERT … TO PUBLIC`, `REVOKE`	3 privileges listed in `user_tab_privs_made`, then 0 after REVOKE
E3	**explicit cursor** `OPEN / FETCH / CLOSE`, `%ISOPEN`, `%NOTFOUND`, `%ROWCOUNT`, `%ROWTYPE`	5 satellites with image counts; "Rows fetched (%ROWCOUNT) = 5"
E4	cursor **FOR UPDATE** + `WHERE CURRENT OF`	MEDIUM alerts with loss ≥ 13% escalated to HIGH (alert 10002, 13.91%), then rolled back
E5	**user-defined exception** `e_too_cloudy` raised with `RAISE`; `%TYPE`; `CONSTANT`	image 23 (Tonle Sap, 3.7% cloud) accepted; image 18 (Mbandaka, 53.3%) rejected: "exceeds 40%"
E6	stand-alone **procedure with IN/OUT parameters**	"Sentinel-2A: 15 images, avg cloud 28%, 61 detected objects"; "Hubble" → ORA-20040 caught
E7	**statement-level trigger**	"3 rows updated, but audit rows added by statement trigger = 1" (a row-level trigger would add 3)
E8	DELETE with **ON DELETE CASCADE** inside SAVEPOINT	objects in image 60: **6 → 0** after "1 row deleted" (the image) → **6** after ROLLBACK
E9	data dictionary `USER_OBJECTS`	FUNCTION 3, INDEX 46, PACKAGE 1, PACKAGE BODY 1, PROCEDURE 1, TABLE 15, TRIGGER 5, VIEW 2

Actual results also observed:

- E1: the 5 satellites listed with mass (Landsat 9 2711 kg … Sentinel-2B 1130 kg); CK_IMG_CLOUD shown **DISABLED** then **ENABLED**; the comment on OBJECT_CHANGE displayed.

- E2: 3 PUBLIC privileges, then `PRIVILEGES_LEFT = 0` after REVOKE.
- E3: 5 satellites; Sentinel-1A shows `images = 0, avg_cloud = n/a`, because the LEFT JOIN keeps a satellite with no images; "Rows fetched (%ROWCOUNT) = 5".
- E9: the **46 indexes** include the 12 explicit FK indexes plus the indexes Oracle creates automatically for every PRIMARY KEY and UNIQUE constraint, and for LOB columns.

> **Issue found during testing (E4):** the first version joined ALERT with OBJECT_CHANGE in the `FOR UPDATE` cursor and used `WHERE CURRENT OF`. It failed with **ORA-01410: invalid ROWID**, because `WHERE CURRENT OF` must identify a single row of a single table. **Fix:** the cursor selects from ALERT only, and the change % is read inside the loop.

10. Transaction Control Demonstrated

Command	Where	Effect shown
`COMMIT`	sample_data, run_procedures, every package procedure	changes made permanent
`SAVEPOINT` / `ROLLBACK TO`	Q19, E8, `pkg_eo.add_detection`	partial undo; the failed detection insert leaves no trace
`ROLLBACK`	E4, E7	demo changes undone, so the dataset stays clean
Atomic multi-row insert	app "Ask" feature (final project)	QUERY_LOG + 5 QUERY_EVIDENCE rows committed together

11. Error Codes Observed and Their Meaning

Error	Cause in this project
ORA-01031	user lacked CREATE VIEW (fixed with GRANT)
ORA-02290	CHECK violation (cloud_cover = 140)
ORA-02291	FK parent missing (image 99999)
ORA-02292	deleting a satellite whose sensors still have images
ORA-20010 / 20020 / 20030 / 20040	custom errors from triggers and procedures
ORA-04098	invalid trigger (the statement-trigger bug described in §6.3)

12. Conclusion

The DA1 design was implemented as 15 constrained tables with 2 views and 12 indexes, loaded with 247 consistent sample detections. A PL/SQL layer (3 functions, a 6-member package, 5 triggers) implements the project's three ideas inside the database:

1. **temporal lineage** (`compute_changes`, Q13)
2. **trust-aware alerting** (`raise_alerts`, Q9/Q11)
3. **confidence propagation** (`fn_query_confidence`)

All outputs matched the independently predicted values. The 19 queries cover joins, aggregates, subqueries, set operators, window functions, hierarchical queries, JSON and transactions. Integrity violations are rejected with clear errors. The same database is extended with a Neo4j graph in DA3 and used by the web application in the final project.